Pop-Up Quiz: Distributional Semantics & Word2Vec

CS115 – Spring 2026

James Pustejovsky

1. **Distributional Hypothesis:** State the distributional hypothesis in your own words and explain why it provides a foundation for vector space semantics.

   Answer: The distributional hypothesis states that words occurring in similar contexts tend to have similar meanings (often summarized as "you shall know a word by the company it keeps"). This provides a foundation for vector space semantics because we can represent words as vectors based on their contextual distributions, where similarity in vector space (e.g., cosine similarity) corresponds to semantic similarity. The hypothesis allows us to learn semantic representations from observable co-occurrence patterns in text without manual semantic annotations.

2. **CBOW vs Skip-gram:** What is the key architectural difference between Continuous Bag of Words (CBOW) and Skip-gram models in word2vec? Which typically performs better for rare words and why?

   Answer: CBOW predicts a target word from its surrounding context words (many-to-one), while Skip-gram predicts context words from a target word (one-to-many). Skip-gram typically performs better for rare words because each occurrence of a rare word generates multiple training examples (one for each context word), providing more learning signal. CBOW, by contrast, treats each target word occurrence as a single training example regardless of context size.

3. **Training Objective:** Write the Skip-gram objective function for word2vec. What is being maximized during training?

   Answer: The Skip-gram objective maximizes the average log probability of context words given target words:

   $$J(\theta) = \frac{1}{T}\sum_{t=1}^{T}\sum_{-c\leq j\leq c, j\neq 0} \log p(w_{t+j} \mid w_t)$$

   where T is the corpus size, c is the context window size, and w_t is the target word. The probability is typically computed using softmax:

   $$p(w_O \mid w_I) = \frac{\exp({\mathbf{v}'_{w_O}}^{\top}\mathbf{v}_{w_I})}{\sum_{w=1}^{W}\exp({\mathbf{v}'_{w}}^{\top}\mathbf{v}_{w_I})}$$

Training maximizes the likelihood that words appearing together in contexts have high dot product similarity in the embedding space.

4. **Negative Sampling:** Explain why negative sampling is used in word2vec training. What computational problem does it solve, and how does it approximate the full softmax?

   Answer: Negative sampling addresses the computational expense of the softmax normalization term, which requires summing over the entire vocabulary (often 100K+ words) for each training example. Instead of computing the full softmax, negative sampling reformulates the problem as binary classification: distinguishing observed word pairs (positive examples) from randomly sampled noise pairs (negative examples). For each positive pair (w_t, w_c), we sample k negative context words from a noise distribution and maximize:

$$\log \sigma({\mathbf{v}'_{w_c}}^\top \mathbf{v}_{w_t}) + \sum_{i=1}^{k} \mathbb{E}_{w_i \sim P_n(w)}[\log \sigma(-{\mathbf{v}'_{w_i}}^\top \mathbf{v}_{w_t})]$$

   This reduces complexity from $O(|V|)$ to $O(k)$ per training example, where typically $k = 5$–20.

5. **Context Windows:** How does the choice of context window size affect the semantic properties captured by word2vec embeddings? Contrast small vs. large windows.

   Answer: Small context windows (2–5 words) tend to capture syntactic and functional similarities, producing embeddings where words are similar if they can substitute for each other in sentences (paradigmatic relations). For example, "Monday" and "Tuesday" become similar. Large context windows (10+ words) capture broader topical and associative relationships (syntagmatic relations), where words that frequently appear in discussions of the same topics become similar. For example, "doctor" and "hospital" might be more associated with larger windows. The window size thus controls the granularity and type of semantic information encoded.

6. **Vector Arithmetic:** The famous analogy "king - man + woman ≈ queen" suggests that word2vec captures certain semantic relationships. What linear algebraic property of the embedding space makes such analogies possible?

   Answer: Word2vec embeddings exhibit linear substructure where semantic relationships correspond to consistent vector offsets. The difference vector $\mathbf{v}_{\text{king}} - \mathbf{v}_{\text{man}}$ captures a "royalty" or "male-to-royal" transformation that is approximately parallel to $\mathbf{v}_{\text{queen}} - \mathbf{v}_{\text{woman}}$. This property emerges because the Skip-gram objective with negative sampling implicitly factorizes a shifted PMI matrix, and similar relational patterns in the data create similar vector geometry. The embedding space thus develops a coordinate system where certain dimensions correspond to semantic features, allowing algebraic operations to compose and manipulate these features.

7. **PMI Connection:** Explain the relationship between word2vec's Skip-gram with Negative Sampling (SGNS) and Pointwise Mutual Information (PMI). What matrix is word2vec implicitly factorizing?

   Answer: Levy and Goldberg (2014) showed that SGNS implicitly performs dimensionality reduction on a shifted PMI matrix. Specifically, the optimal Skip-gram solution with negative sampling minimizes a loss function that is equivalent to factorizing a matrix $\mathbf{M}$ where:

   $$M_{ij} = \text{PMI}(w_i, w_j) - \log k$$

   where k is the number of negative samples. The word and context embeddings are low-rank factors that approximately reconstruct this shifted PMI matrix. This connection provides theoretical insight: word2vec learns to represent which word pairs co-occur more than expected by chance (positive PMI) while the shift accounts for the sampling of negative examples.

8. **Subsampling:** Word2vec uses subsampling to randomly remove frequent words during training. Why is this beneficial for learning high-quality embeddings?

   Answer: Subsampling addresses the imbalance between frequent and infrequent words. Very frequent words (like "the", "is", "of") appear in many contexts but often carry little semantic content, and their overwhelming presence can dominate training and obscure more meaningful patterns. By probabilistically discarding frequent words (with probability proportional to their frequency), subsampling: (1) speeds up training, (2) balances the representation of rare vs. common words, and (3) allows the model to better capture semantic relationships among content words rather than just syntactic function patterns. The subsampling probability is typically $P(\text{discard_}w) = 1 - \sqrt{t/f(w)}$ where $f(w)$ is word frequency and t is a threshold (commonly 10^{-5}).

9. **Out-of-Vocabulary:** What is a fundamental limitation of word2vec regarding out-of-vocabulary (OOV) words? How do subword-based approaches like fastText address this issue?

   Answer: Word2vec maintains a fixed vocabulary learned during training and cannot generate embeddings for OOV words—words not seen during training or misspellings. This is because each word is treated as an atomic unit with its own embedding vector. FastText addresses this by representing words as bags of character n-grams (e.g., "where" becomes $\langle$wh, whe, her, ere, re$\rangle$ plus $\langle$where$\rangle$). The word embedding is the sum of its n-gram embeddings. For OOV words, fastText can compose an embedding from known character n-grams, enabling generalization to new words and robustness to spelling variations. This is particularly valuable for morphologically rich languages.

10. **Evaluation:** Name two common intrinsic evaluation methods for word embeddings and one extrinsic evaluation approach. What are the limitations of relying solely on intrinsic evaluations?

Answer: Common intrinsic evaluations include: (1) **Word similarity tasks** where embedding cosine similarities are correlated with human similarity judgments (e.g., WordSim-353, SimLex-999), and (2) **Analogy tasks** where the model solves proportional analogies like "king:man::queen:?" using vector arithmetic. An extrinsic evaluation embeds word2vec vectors into downstream tasks like **text classification, named entity recognition, or sentiment analysis** and measures task performance.

The limitation of intrinsic evaluations is that high performance doesn't guarantee utility for actual applications. Different tasks may require different semantic properties, and word similarity datasets are often small, domain-specific, and may not reflect the semantic distinctions needed for particular applications. Extrinsic evaluation on target tasks provides more reliable evidence of embedding quality for intended use cases.